

ColorVisionApp Documentation

1. Various States an App Can Enter on React Native

On the React Native platform, an app can enter the following states:

- **Active:** The app is running in the foreground, is fully visible to the user, and is actively receiving user input. This is the primary state for most app interactions.
- **Inactive:** The app is transitioning between states, typically when interrupted by a transient event (e.g., an incoming call or notification overlay). It is in the foreground but not receiving input.
- **Background:** The app is running but not visible to the user. It may still perform certain limited background tasks, such as saving data, processing asynchronous tasks, or maintaining a timer, if permitted by the system.
- **Suspended (Tombstoned):** The app is no longer active or in memory, but the system keeps a record of its last state. In low-memory situations, apps may be suspended and purged by the OS, which requires restoring the app state upon return to provide continuity for the user.
- **Closed/Terminated:** The app is fully terminated, with no data retained in memory. When reopened, the app starts fresh as though it was just launched. Any saved data from previous sessions (e.g., persistent settings) must be reloaded from storage.

2. Specific States for ColorVisionApp, Justification, and Required Behavior

For ColorVisionApp, the following states are essential to consider to maintain a consistent and seamless user experience.

a. Active State

- **Why Consider It:** This is the primary state where users interact with the app, whether logging in, taking color vision tests, or editing profile settings. All key app functions (navigation, data entry, and real-time feedback) occur in this state.
- **Required Behavior:**
 - The app should load necessary data (user preferences, profile information) and display all UI elements appropriately.
 - All input fields (like age, gender, bio) should be available, and buttons should trigger their associated actions immediately (e.g., saving data or navigating to another screen).

b. Background State

- **Why Consider It:** Users may temporarily leave the app to attend to other tasks (e.g., checking a message or answering a call). When they return, they should be able to pick up where they left off without losing any data.
- **Required Behavior:**
 - The app should save critical state information (e.g., quiz progress, unsaved profile updates, or user preferences like Dark Mode and High Contrast settings) to AsyncStorage before moving to the background.
 - When the app returns to the active state, it should restore these saved settings, ensuring continuity in the user experience.

c. Suspended State (Tombstoned)

- **Why Consider It:** In low-memory situations, the OS may suspend ColorVisionApp. Managing this state ensures that users return to the same point in the app without data loss, even after a suspension.
- **Required Behavior:**

- The app should save the current state (such as test progress, last active screen, or user preferences) to persistent storage when entering the background state to prepare for potential suspension.
- Upon resuming from suspension, the app should restore the saved state, including loading the appropriate screen (e.g., if a user was in the middle of a test) and applying settings like Dark Mode and High Contrast.

d. Closed/Terminated State

- **Why Consider It:** When users terminate the app, they expect their preferences and profile settings to be retained for the next session.
- **Required Behavior:**
 - The app should store persistent settings (e.g., user profile information, Dark Mode, and High Contrast preferences) in AsyncStorage so that they can be loaded the next time the app is launched.
 - Upon reopening, the app should start from the login or home screen but should automatically apply previously saved settings, ensuring that user preferences are respected even if the app was fully terminated.